

Claude Code 最佳实践

修复：狭隘地限定调查或使用 subagents，以便探索不会消耗你的主 context。

DESIGNED HTML FIRST · PRINTED TO PDF WITH CHROME

source	examples/zh-cn-best-practices/best-practices.md
lines	606
sections	10
code blocks	17

Section map

01 给 Claude 一种验证其工作的方式

02 先探索，再规划，最后编码

03 在提示中提供具体的上下文

04 配置你的环境

05 有效沟通

06 管理你的会话

07 自动化和扩展

08 避免常见失败模式

09 培养你的直觉

10 相关资源

Documentation Index

Fetch the complete documentation index at:
<https://code.claude.com/docs/llms.txt>

Use this file to discover all available pages before exploring further.

从配置环境到跨并行会话扩展，充分利用 Claude Code 的提示和模式。

Claude Code 是一个代理式编码环境。与等待回答问题的聊天机器人不同，Claude Code 可以读取你的文件、运行命令、进行更改，并在你观看、重定向或完全离开的情况下自主解决问题。

这改变了你的工作方式。与其自己编写代码并要求 Claude 审查，不如描述你想要什么，让 Claude 弄清楚如何构建它。Claude 会探索、规划和实现。

但这种自主性仍然伴随着学习曲线。Claude 在某些约束条件下工作，你需要理解这些约束。

本指南涵盖了在 Anthropic 内部团队和在各种代码库、语言和环境中使用 Claude Code 的工程师中已被证明有效的模式。有关代理循环如何在幕后工作的信息，请参阅 [Claude Code 如何工作](#)。

大多数最佳实践都基于一个约束：Claude 的 context window 填充速度很快，随着填充，性能会下降。

Claude 的 context window 保存你的整个对话，包括每条消息、Claude 读取的每个文件和每个命令输出。但这可能会很快填满。单个调试会话或代码库探索可能会生成并消耗数万个 token。

这很重要，因为当 context 填充时，LLM 性能会下降。当 context window 即将满时，Claude 可能会开始“遗忘”早期的指令或犯更多错误。context window 是最重要的资源。要查看会话在实践中如何填充，请 [观看交互式演练](#)，了解启动时加载的内容以及每个文件读取的成本。使用 [自定义状态行](#) 持续跟踪 context 使用情况，并查看 [减少 token 使用](#) 了解减少 token 使用的策略。

给 Claude 一种验证其工作的方式

给 Claude 一个它可以运行的检查：测试、构建、屏幕截图进行比较。这是你观看的会话和你可以离开的会话之间的区别。

当工作看起来完成时，Claude 会停止。没有它可以运行的检查，“看起来完成”是唯一可用的信号，你成为验证循环：每个错误都在等待你注意到它。给 Claude 一些能产生通过或失败的东西，循环就会自动关闭。Claude 完成工作，运行检查，读取结果，并迭代直到检查通过。

检查是任何返回 Claude 可以在对话中读取的信号的东西：测试套件、构建退出代码、linter、针对固定装置比较输出的脚本，或与设计进行比较的[浏览器屏幕截图](#)。

策略	之前	之后
提供验证标准	"实现一个验证电子邮件地址的函数"	"编写一个 validateEmail 函数。示例测试用例：user@example.com 为真，invalid 为假，user@.com 为假。实现后运行测试"
以视觉方式验证 UI 更改	"让仪表板看起来更好"	"\[粘贴屏幕截图] 实现此设计。对结果进行屏幕截图并与原始设计进行比较。列出差异并修复它们"
解决根本原因，而不是症状	"构建失败"	"构建失败，出现此错误：\[粘贴错误]。修复它并验证构建成功。解决根本原因，不要抑制错误"

一旦检查存在，决定它对停止的限制有多严格：

- 在一个提示中：要求 Claude 运行检查并在同一消息中迭代，如上表所示。
- 在整个会话中：将检查设置为 `/goal` 条件。单独的评估器在每次转换后重新检查它，Claude 继续工作直到它成立。
- 作为确定性门：Stop hook 作为脚本运行你的检查，并阻止转换结束直到它通过。Claude Code 覆盖 hook 并在 8 次连续阻止后结束转换。
- 通过第二意见：验证子代理或动态工作流检查自己的发现，有一个新鲜的模型尝试反驳结果，所以做工作的代理不是给它评分的。

每一步都用设置换取关注。提示版本适用于今天的任何任务。`/goal` 和 Stop hook 版本是让无人值守运行正确完成而无需你的东西。

让 Claude 显示证据而不是声称成功：测试输出、它运行的命令及其返回的内容，或结果的屏幕截图。审查证据比自己重新运行验证要快，并且它适用于你没有观看的会话。

先探索，再规划，最后编码

将研究和规划与实现分开，以避免解决错误的问题。

让 Claude 直接跳到编码可能会产生解决错误问题的代码。使用 **Plan Mode** 将探索与执行分开。

推荐的工作流有四个阶段：

01 探索

进入 Plan Mode。Claude 读取文件并回答问题，不进行任何更改。

```
read /src/auth and understand how we handle sessions and login.  
also look at how we manage environment variables for secrets.
```

02 规划

要求 Claude 创建详细的实现计划。

```
I want to add Google OAuth. What files need to change?  
What's the session flow? Create a plan.
```

按 **Ctrl+G** 在文本编辑器中打开计划进行直接编辑，然后 Claude 继续。

03 实现

切换出 Plan Mode 并让 Claude 编码，根据其计划进行验证。

```
implement the OAuth flow from your plan. write tests for the  
callback handler, run the test suite and fix any failures.
```

要求 Claude 使用描述性消息进行提交并创建 PR。

```
commit with a descriptive message and open a PR
```

Plan Mode 很有用，但也增加了开销。

对于范围明确且修复很小的任务（如修复拼写错误、添加日志行或重命名变量），要求 Claude 直接执行。

当你对方法不确定、更改修改多个文件或你不熟悉被修改的代码时，规划最有用。如果你能用一句话描述 diff，跳过计划。

在提示中提供具体的上下文

你的指令越精确，你需要的更正就越少。

Claude 可以推断意图，但它不能读心术。引用特定文件、提及约束，并指出示例模式。

策略	之前	之后
限定任务范围。指定哪个文件、什么场景和测试偏好。	"为foo.py 添加测试"	"为foo.py 编写测试，涵盖用户已注销的边界情况。避免 mock。"
指向来源。指导 Claude 到可以回答问题的来源。	"为什么 ExecutionFactory y 有这样奇怪的 api? "	"查看 ExecutionFactory 的 git 历史并总结其 api 是如何形成的"
参考现有模式。指向代码库中的模式。	"添加日历小部件"	"查看主页上现有小部件的实现方式以了解模式。HotDogWidget.php 是一个很好的例子。按照模式实现一个新的日历小部件，让用户选择月份并向前/向后分页以选择年份。从头开始构建，除了代码库中已使用的库外，不使用其他库。"
描述症状。提供症状、可能的位置以及"修复"的样子。	"修复登录错误"	"用户报告会超时后登录失败。检查src/auth/ 中的身份验证流程，特别是 token 刷新。编写一个失败的测试来重现问题，然后修复它"

当你在探索并能够改正方向时，模糊的提示可能很有用。像 "你会改进这个文件的什么? " 这样的提示可以表面你不会想到要问的东西。

提供丰富的内容

使用 @ 引用文件、粘贴屏幕截图/图像或直接管道数据。

你可以通过多种方式向 Claude 提供丰富的数据：

- 使用 @ 引用文件，而不是描述代码的位置。Claude 在响应前读取文件。
- 直接粘贴图像。复制/粘贴或拖放图像到提示中。
- 提供 URL 用于文档和 API 参考。使用 /permissions 来允许列表经常使用的域。
- 管道数据 通过运行 cat error.log | claude 直接发送文件内容。
- 让 Claude 获取它需要的东西。告诉 Claude 使用 Bash 命令、MCP 工具或通过读取文件来自己拉取上下文。

配置你的环境

一些设置步骤使 Claude Code 在所有会话中显著更有效。有关扩展功能的完整概述和何时使用每个功能，请参阅 [扩展 Claude Code](#)。

编写有效的 CLAUDE.md

运行 `/init` 根据你的当前项目结构生成启动 CLAUDE.md 文件，然后随时间精化。

CLAUDE.md 是一个特殊文件，Claude 在每次对话开始时读取。包括 Bash 命令、代码风格和工作流规则。这给 Claude 提供了它无法从代码中推断的持久上下文。

`/init` 命令分析你的代码库以检测构建系统、测试框架和代码模式，为你提供坚实的基础来精化。

CLAUDE.md 文件没有必需的格式，但保持简短和易读。例如：

```
# Code style
- Use ES modules (import/export) syntax, not CommonJS (require)
- Destructure imports when possible (eg. import { foo } from 'bar')

# Workflow
- Be sure to typecheck when you're done making a series of code changes
- Prefer running single tests, and not the whole test suite, for performance
```

CLAUDE.md 在每个会话中加载，所以只包括广泛适用的东西。对于仅有时相关的域知识或工作流，改用 [skills](#)。Claude 按需加载它们，不会使每次对话都膨胀。

保持简洁。对于每一行，问自己：“删除这个会导致 Claude 犯错吗？” 如果不会，删除它。膨胀的 CLAUDE.md 文件会导致 Claude 忽略你的实际指令！

✅ 包括	❌ 排除
Claude 无法猜测的 Bash 命令	Claude 可以通过读取代码弄清楚的任何东西
与默认值不同的代码风格规则	Claude 已经知道的标准语言约定
测试指令和首选测试运行器	详细的 API 文档（改为链接到文档）
存储库礼仪（分支命名、PR 约定）	经常变化的信息
特定于你的项目的架构决策	长解释或教程
开发者环境怪癖（必需的环境变量）	自明的实践，如“编写干净的代码”
常见陷阱或非显而易见的行为	文件逐个描述代码库

如果 Claude 继续做你不想要的事情，尽管有反对的规则，该文件可能太长，规则被遗漏了。如果 Claude 问你在 CLAUDE.md 中回答的问题，措辞可能不明确。像对待代码一样对待 CLAUDE.md：当事情出错时审查它，定期修剪它，并通过观察 Claude 的行为是否实际改变来测试更改。

你可以通过添加强调（例如"IMPORTANT"或"YOU MUST"）来调整指令以改进遵守。将文件检入 git，以便你的团队可以贡献。该文件随时间增加价值。

CLAUDE.md 文件可以使用 `@path/to/import` 语法导入其他文件：

```
See @README.md for project overview and @package.json for available npm commands.

# Additional Instructions
- Git workflow: @docs/git-instructions.md
- Personal overrides: @~/my-project-instructions.md
```

你可以在多个位置放置 CLAUDE.md 文件：

- 主文件夹（`~/my-project/CLAUDE.md`）：适用于所有 Claude 会话
- 项目根目录（`./CLAUDE.md`）：检入 git 以与你的团队共享
- 项目根目录（`./CLAUDE.local.md`）：个人项目特定的笔记；将此文件添加到你的 `.gitignore`，以便它不会与你的团队共享
- 父目录：对于 monorepos 有用，其中 `root/CLAUDE.md` 和 `root/foo/CLAUDE.md` 都会自动拉入
- 子目录：当处理这些目录中的文件时，Claude 按需拉入子 CLAUDE.md 文件

配置权限

使用 `auto mode` 让分类器处理批准，使用 `/permissions` 来允许列表特定命令，或使用 `/sandbox` 进行操作系统级隔离。每种方式都减少中断，同时让你保持控制。

默认情况下，Claude Code 请求可能修改你的系统的操作的权限：文件写入、Bash 命令、MCP 工具等。这是安全的但繁琐。在第十次批准后，你不是真的在审查，你只是点击通过。有三种方式来减少这些中断：

- Auto mode**：一个单独的分类器模型审查命令并仅阻止看起来有风险的东西：范围升级、未知基础设施或由敌对内容驱动的操作。最适合当你信任任务的总体方向但不想点击通过每一步时
- 权限允许列表**：允许你知道是安全的特定工具，如 `npm run lint` 或 `git commit`
- 沙箱**：启用操作系统级隔离，限制文件系统和网络访问，允许 Claude 在定义的边界内更自由地工作

阅读更多关于 [权限模式](#)、[权限规则](#) 和 [沙箱](#)。

使用 CLI 工具

告诉 Claude Code 在与外部服务交互时使用 CLI 工具，如 `gh`、`aws`、`gcloud` 和 `sentry-cli`。

CLI 工具是与外部服务交互的最 context 高效的方式。如果你使用 GitHub，安装 `gh` CLI。Claude 知道如何使用它来创建问题、打开拉取请求和读取评论。没有 `gh`，Claude 仍然可以使用 GitHub API，但未认证的请求经常会触发速率限制。

Claude 也有效地学习它不知道的 CLI 工具。尝试像 `Use 'foo-cli-tool --help' to learn about foo tool, then use it to solve A, B, C.` 这样的提示。

连接 MCP 服务器

运行 `claude mcp add` 来连接外部工具，如 Notion、Figma 或你的数据库。

使用 [MCP servers](#)，你可以要求 Claude 从问题跟踪器实现功能、查询数据库、分析监控数据、集成来自 Figma 的设计并自动化工作流。

设置 hooks

使用 hooks 来处理必须每次发生且没有例外的操作。

[Hooks](#) 在 Claude 工作流中的特定点自动运行脚本。与 `CLAUDE.md` 指令不同，hooks 是确定性的，保证操作发生。

Claude 可以为你编写 hooks。尝试像 "编写一个在每次文件编辑后运行 `eslint` 的 hook" 或 "编写一个阻止写入迁移文件夹的 hook" 这样的提示。编辑 `.claude/settings.json` 直接配置 hooks，并运行 `/hooks` 来浏览配置的内容。

创建 skills

在 `.claude/skills/` 中创建 `SKILL.md` 文件，为 Claude 提供域知识和可重用工作流。

[Skills](#) 使用特定于你的项目、团队或域的信息扩展 Claude 的知识。Claude 在相关时自动应用它们，或者你可以使用 `/skill-name` 直接调用它们。

通过向 `.claude/skills/` 添加带有 `SKILL.md` 的目录来创建 skill：

```
---
name: api-conventions
description: REST API design conventions for our services
---

# API Conventions

- Use kebab-case for URL paths
- Use camelCase for JSON properties
- Always include pagination for list endpoints
- Version APIs in the URL path (/v1/, /v2/)
```

Skills 也可以定义你直接调用的可重复工作流：

```
---
name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true
---

Analyze and fix the GitHub issue: $ARGUMENTS.

1. Use `gh issue view` to get the issue details
2. Understand the problem described in the issue
3. Search the codebase for relevant files
4. Implement the necessary changes to fix the issue
5. Write and run tests to verify the fix
6. Ensure code passes linting and type checking
7. Create a descriptive commit message
8. Push and create a PR
```

运行 `/fix-issue 1234` 来调用它。对于具有你想手动触发的副作用的工作流，使用 `disable-model-invocation: true`。

创建自定义 subagents

在 `.claude/agents/` 中定义专门的助手，Claude 可以委托给它们来处理隔离的任务。

Subagents 在自己的 context 中运行，拥有自己的一组允许的工具。它们对于读取许多文件或需要专门关注而不会使你的主对话混乱的任务很有用。

```
---
name: security-reviewer
description: Reviews code for security vulnerabilities
tools: Read, Grep, Glob, Bash
model: opus
---

You are a senior security engineer. Review code for:

- Injection vulnerabilities (SQL, XSS, command injection)
- Authentication and authorization flaws
- Secrets or credentials in code
- Insecure data handling

Provide specific line references and suggested fixes.
```

明确告诉 Claude 使用 subagents: "使用 subagent 来审查此代码的安全问题。"

安装 plugins

运行 `/plugin` 来浏览市场。Plugins 添加 skills、工具和集成，无需配置。

Plugins 将 skills、hooks、subagents 和 MCP 服务器捆绑到来自社区和 Anthropic 的单个可安装单元中。如果你使用类型化语言，安装 [代码智能 plugin](#) 来为 Claude 提供精确的符号导航和编辑后的自动错误检测。

有关在 skills、subagents、hooks 和 MCP 之间选择的指导，请参阅 [扩展 Claude Code](#)。

有效沟通

你与 Claude Code 沟通的方式显著影响结果的质量。

提出代码库问题

问 Claude 你会问资深工程师的问题。

当加入新代码库时，使用 Claude Code 进行学习和探索。你可以问 Claude 你会问另一个工程师的相同类型的问题：

- 日志如何工作？
- 我如何创建新的 API 端点？
- `foo.rs` 第 134 行的 `async move { ... }` 做什么？
- `CustomerOnboardingFlowImpl` 处理哪些边界情况？
- 为什么这段代码在第 333 行调用 `foo()` 而不是 `bar()`？

以这种方式使用 Claude Code 是一个有效的入职工作流，改进了加入时间并减少了对其他工程师的负担。无需特殊提示：直接提问。

让 Claude 采访你

对于更大的功能，让 Claude 先采访你。从最小的提示开始，要求 Claude 使用 `AskUserQuestion` 工具采访你。

Claude 会问你可能还没有考虑过的东西，包括技术实现、UI/UX、边界情况和权衡。

```
I want to build [brief description]. Interview me in detail using the AskUserQuestion tool.
```

```
Ask about technical implementation, UI/UX, edge cases, concerns, and tradeoffs. Don't ask obvious questions, dig into the hard parts I might not have considered.
```

```
Keep interviewing until we've covered everything, then write a complete spec to SPEC.md.
```

一旦规范完成，启动新会话来执行它。新会话有干净的 context，完全专注于实现，你有一个书面规范可以参考。

最有用的规范是自包含的：它们命名涉及的文件和接口，说明什么在范围之外，并以端到端验证步骤结束，证明该功能有效。花在使规范精确上的时间比花在观看实现上的时间收益更大。

管理你的会话

对话是持久的和可逆的。利用这一点！

尽早且经常改正方向

一旦你注意到 Claude 偏离轨道，立即改正它。

最好的结果来自紧密的反馈循环。虽然 Claude 有时会在第一次尝试时完美地解决问题，但快速改正它通常会更快地产生更好的解决方案。

- **Esc**：使用 **Esc** 键在中途停止 Claude。Context 被保留，所以你可以重定向。
- **Esc + Esc** 或 **/rewind**：按 **Esc** 两次或运行 **/rewind** 来打开 rewind 菜单并恢复之前的对话和代码状态，或从选定的消息进行总结。
- **"撤销那个"**：让 Claude 恢复其更改。
- **/clear**：在不相关的任务之间重置 context。长会话与无关的 context 可能会降低性能。

如果你在一个对话中对同一问题改正了 Claude 两次以上，context 就充满了失败的方法。运行 **/clear** 并使用更具体的提示重新开始，该提示包含你学到的东西。干净的会话与更好的提示几乎总是优于长会话与累积的改正。

积极管理 context

在不相关的任务之间频繁运行 **/clear** 来重置 context。

Claude Code 在你接近 context 限制时自动压缩对话历史，这保留了重要的代码和决策，同时释放空间。

在长会话中，Claude 的 context window 可能会充满无关的对话、文件内容和命令。这可能会降低性能，有时会分散 Claude 的注意力。

- 在任务之间频繁使用 **/clear** 来完全重置 context window
- 当自动压缩触发时，Claude 总结最重要的东西，包括代码模式、文件状态和关键决策
- 为了更多控制，运行 **/compact <instructions>**，如 **/compact Focus on the API changes**
- 要仅压缩对话的一部分，使用 **Esc + Esc** 或 **/rewind**，选择消息检查点，并选择 **从这里总结** 或 **总结到这里**。第一个会压缩从该点开始的消息，同时保持早期 context 完整；第二个会压缩早期消息，同时保持最近的消息完整。请参阅 [恢复与总结](#)。
- 在 CLAUDE.md 中使用像 **"When compacting, always preserve the full list of modified files and any test commands"** 这样的指令来自定义压缩行为，以确保关键 context 在总结中存活
- 对于不需要留在 context 中的快速问题，使用 **/btw**。答案出现在可关闭的覆盖层中，永远不会进入对话历史，所以你可以检查细节而不增加 context。

使用 subagents 进行调查

使用 `"use subagents to investigate X"` 委托研究。它们在单独的 context 中探索，为实现保持你的主对话干净。

由于 context 是你的基本约束，subagents 是可用的最强大的工具之一。当 Claude 研究代码库时，它读取许多文件，所有这些都消耗你的 context。Subagents 在单独的 context windows 中运行并报告摘要：

```
Use subagents to investigate how our authentication system handles token
refresh, and whether we have any existing OAuth utilities I should reuse.
```

subagent 探索代码库、读取相关文件并报告发现，所有这些都不会使你的主对话混乱。

你也可以在 Claude 实现某些东西后使用 subagents 进行验证：

```
use a subagent to review this code for edge cases
```

使用检查点进行 Rewind

Claude 进行的每个提示都会创建一个检查点。你可以将对话、代码或两者恢复到任何之前的检查点。

Claude 在每次更改前自动对文件进行快照，以便检查点可以恢复它们。双击 `Escape` 或运行 `/rewind` 来打开 rewind 菜单。你可以仅恢复对话、仅恢复代码、恢复两者或从选定的消息进行总结。有关详细信息，请参阅 [Checkpointing](#)。

与其仔细规划每一步，你可以告诉 Claude 尝试一些冒险的事情。如果不起作用，rewind 并尝试不同的方法。检查点在会话中持续，所以你可以关闭你的终端并稍后仍然 rewind。

<Warning> 检查点仅跟踪 Claude 进行的更改，不跟踪外部进程。这不是 git 的替代品。</Warning>

恢复对话

使用 `/rename` 给会话命名，并像对待分支一样对待它们：每个工作流都有自己的持久 context。

Claude Code 在本地保存对话，所以当任务跨越多个会话时，你不必重新解释 context。运行 `claude --continue` 来继续最近的会话，或 `claude --resume` 来从列表中选择。给会话起描述性名称，如 `oauth-migration`，以便你稍后可以找到它们。有关完整的恢复、分支和命名控制集，请参阅 [管理会话](#)。

自动化和扩展

一旦你对一个 Claude 有效，通过并行会话、非交互模式和扇出模式来增加你的输出。

到目前为止，一切都假设一个人、一个 Claude 和一个对话。但 Claude Code 水平扩展。本部分中的技术展示了你如何能做更多。

运行非交互模式

在 CI、pre-commit hooks 或脚本中使用 `claude -p "prompt"`。添加 `--output-format stream-json --verbose` 用于流式 JSON 输出。

使用 `claude -p "your prompt"`，你可以非交互地运行 Claude，不需要会话。非交互模式是你将 Claude 集成到 CI 管道、pre-commit hooks 或任何自动化工作流中的方式。输出格式让你以编程方式解析结果：纯文本、JSON 或流式 JSON。

```
# One-off queries
claude -p "Explain what this project does"

# Structured output for scripts
claude -p "List all API endpoints" --output-format json

# Streaming for real-time processing
claude -p "Analyze this log file" --output-format stream-json --verbose
```

运行多个 Claude 会话

并行运行多个 Claude 会话以加快开发、运行隔离的实验或启动复杂的工作流。

选择适合你想要自己进行多少协调的并行方法：

- **Worktrees**：在隔离的 git 检出中运行单独的 CLI 会话，以便编辑不会冲突
- **桌面应用**：以视觉方式管理多个本地会话，每个会话都在自己的 worktree 中
- **Claude Code 在网上**：在 Anthropic 管理的云基础设施中的隔离虚拟机上运行会话
- **Agent teams**：具有共享任务、消息和团队主管的多个会话的自动协调

除了并行化工作，多个会话启用了质量关注的工作流。新鲜的 context 改进了代码审查，因为 Claude 不会偏向于它刚刚编写的代码。

例如，使用 Writer/Reviewer 模式：

会话 A (Writer)	会话 B (Reviewer)
为我们的 API 端点实现速率限制器	
	审查 @src/middleware/rateLimiter.ts 中的速率限制器实现。查找边界情况、竞态条件和与我们现有中间件模式的一致性。
这是审查反馈：[会话 B 输出]。解决这个问题。	

你可以用测试做类似的事情：让一个 Claude 编写测试，然后另一个编写代码来通过它们。

跨文件扇出

循环遍历任务，为每个调用 `claude -p`。使用 `--allowedTools` 来限定批量操作的权限。

对于大型迁移或分析，你可以跨许多并行 Claude 调用分配工作：

01
生成任务列表

让 Claude 列出所有需要迁移的文件（例如，`list all 2,000 Python files that need migrating`）

02
编写脚本来循环遍历列表

```
for file in $(cat files.txt); do
  claude -p "Migrate $file from React to Vue. Return OK or FAIL." \
    --allowedTools "Edit,Bash(git commit *)"
done
```

03
在几个文件上测试，然后大规模运行

根据前 2-3 个文件出错的情况精化你的提示，然后在完整集合上运行。`--allowedTools` 标志限制 Claude 能做什么，这在你无人值守运行时很重要。

你也可以将 Claude 集成到现有的数据/处理管道中：

```
claude -p "<your prompt>" --output-format json | your_command
```

在开发期间使用 `--verbose` 进行调试，在生产中关闭它。

使用 auto mode 自主运行

为了不间断的执行和后台安全检查，使用 `auto mode`。分类器模型在命令运行前审查它们，阻止范围升级、未知基础设施和由敌对内容驱动的操作，同时让常规工作无提示进行。

```
claude --permission-mode auto -p "fix all lint errors"
```

对于使用 `-p` 标志的非交互运行，如果分类器重复阻止操作，`auto mode` 会中止，因为没有用户可以回退到。请参阅 [auto mode 何时回退](#) 了解阈值。

添加对抗性审查步骤

在将任务视为完成之前，让一个子代理在新鲜的 context 中审查差异并报告缺陷。

Claude 无人值守工作的时间越长，在你将工作视为完成之前进行独立检查就越重要。在新鲜的 `subagent` context 中运行的审查者只看到差异和你给它的标准，而不是产生更改的推理，所以它按自己的条件评估结果。

对于正确性检查，运行捆绑的 `/code-review` skill，它在新鲜的子代理中审查当前差异以查找错误，并将发现返回到会话。要检查差异是否符合你的计划，请自己编写审查提示。命名要检查的工作、要检查的计划以及什么算作发现：

使用子代理根据 `PLAN.md` 审查速率限制器差异。检查每个要求是否已实现、列出的边界情况是否有测试，以及任务范围之外是否有任何更改。报告缺陷，而不是风格偏好。

因为审查者作为子代理运行，实现会话直接接收缺陷，可以修复它们并重新审查，而无需你在窗口之间复制发现。对于更长的自主运行，`agent team` 可以在许多任务中保持这个循环进行，而你可以对记录的发现进行抽查。

被提示查找缺陷的审查者通常会报告一些，即使工作是健全的，因为那是它被要求做的。追逐每个发现会导致过度工程：额外的抽象层、防御性代码和针对无法发生的情况的测试。告诉审查者只标记影响正确性或陈述要求的缺陷，将其余的视为可选。

避免常见失败模式

这些是常见的错误。尽早识别它们可以节省时间：

- 厨房水槽会话。你从一个任务开始，然后问 Claude 一些不相关的东西，然后回到第一个任务。Context 充满了无关的信息。

修复：在不相关的任务之间 `/clear`。

- 一次又一次地改正。Claude 做错了什么，你改正它，它仍然是错的，你再改正。Context 被失败的方法污染。

修复：在两次失败的改正后，`/clear` 并编写一个更好的初始提示，包含你学到的东西。

- 过度指定的 **CLAUDE.md**。如果你的 CLAUDE.md 太长，Claude 会忽略一半，因为重要的规则在噪音中丢失。

修复：无情地修剪。如果 Claude 已经在没有指令的情况下正确地做某事，删除它或将其转换为 hook。

- 信任然后验证的差距。Claude 产生一个看起来合理的实现，但不处理边界情况。

修复：始终提供验证（测试、脚本、屏幕截图）。如果你不能验证它，不要发布它。

- 无限探索。你要求 Claude "调查"某些东西而不限定范围。Claude 读取数百个文件，填充 context。

修复：狭隘地限定调查或使用 subagents，以便探索不会消耗你的主 context。

培养你的直觉

本指南中的模式不是一成不变的。它们是通常效果很好的起点，但可能不是每种情况的最优选择。

有时你_应该_让 context 累积，因为你深入一个复杂的问题，历史很有价值。有时你应该跳过规划，让 Claude 弄清楚，因为任务是探索性的。有时模糊的提示正是你想要的，因为你想看看 Claude 如何解释问题，然后再限制它。

注意什么有效。当 Claude 产生很好的输出时，注意你做了什么：提示结构、你提供的 context、你所在的模式。当 Claude 遇到困难时，问为什么。Context 太嘈杂了吗？提示太模糊了吗？任务对于一次通过来说太大了吗？

随着时间的推移，你会培养没有指南能捕捉的直觉。你会知道何时具体，何时开放，何时规划，何时探索，何时清除 context，何时让它累积。

相关资源

- [Claude Code 如何工作](#)：代理循环、工具和 context 管理
- [扩展 Claude Code](#)：skills、hooks、MCP、subagents 和 plugins
- [常见工作流](#)：调试、测试、PR 等的分步配方
- [CLAUDE.md](#)：存储项目约定和持久 context